arXiv:2211.14657v3 [cond-mat.dis-nn] 17 Aug 2024

Variational Tensor Neural Networks for Deep Learning

Saeed S. Jahromi[1,2,3] **and Román Orús**[2,3,4,*]

[1]Department of Physics, Institute for Advanced Studies in Basic Sciences (IASBS), Zanjan 45137-66731, Iran
[2]Donostia International Physics Center, Paseo Manuel de Lardizabal 4, E-20018 San Sebastián, Spain
[3]Multiverse Computing, Paseo de Miramón 170, E-20014 San Sebastián, Spain
[4]Ikerbasque Foundation for Science, Maria Diaz de Haro 3, E-48013 Bilbao, Spain
[*]roman.orus@dipc.org

ABSTRACT

Deep neural networks (NNs) encounter scalability limitations when confronted with a vast array of neurons, thereby constraining their achievable network depth. To address this challenge, we propose an integration of tensor networks (TN) into NN frameworks, combined with a variational DMRG-inspired training technique. This in turn, results in a scalable tensor neural network (TNN) architecture capable of efficient training over a large parameter space. Our variational algorithm utilizes a local gradient-descent technique, enabling manual or automatic computation of tensor gradients, facilitating design of hybrid TNN models with combined dense and tensor layers. Our training algorithm further provides insight on the entanglement structure of the tensorized trainable weights and correlation among the model parameters. We validate the accuracy and efficiency of our method by designing TNN models and providing benchmark results for linear and non-linear regressions, data classification and image recognition on MNIST handwritten digits.

Introduction

Machine learning algorithms based on deep learning have demonstrated remarkable success in a diverse range of tasks, including identification[1–3], classification[4–6], regression[7,8], clustering[9–11], and numerous other artificial intelligence applications. Deep learning algorithms, particularly those employing neural networks (NNs), have garnered significant attention due to their enhanced capabilities in feature learning[12,13] and decision-making[14,15], both in supervised and unsupervised learning scenarios. In addition, NNs have also found practical applications in condensed matter and statistical physics[16–18]. As an example, it has been shown that NNs have expressive power for representing complex many-body wave functions[19–23]. Moreover, ideas for detecting phase transitions in quantum many-body systems with fully connected NNs and convolutional neural networks (CNN) have also been successfully put forward[24–27].

Recently, the connection between tensor networks (TN), which are efficient ansatz for representing quantum many-body wave functions[28,29], and neural networks, has also been established. It has been shown that the trainable weights of NNs are closely related to many-body wave functions, henceforth can be replaced by TNs and trained using variational optimization techniques[30–33]. Efficient TN algorithms for classification[30], anomaly detection[34], and clustering[35] have been proposed. On top of their expressive power, TNs also provide efficient schemes for data compression based on tensor factorization[36–39]. As an example, the number of parameters in NN models can substantially be compressed by keeping only the most relevant degrees of freedom by discarding those that involve lower correlations. Tensor neural networks (TNN)[31] and tensor convolutional neural networks (TCNN)[40] are examples of NNs where the weight tensors of the hidden layers are replaced by tensor network structures using, e.g., the singular value decomposition (SVD). Recent studies actually confirm that, for such a reduced parameter space, TNNs have better performance and accuracy than standard NNs[31,36].

In most of the current TNN developments, the tensorization takes place only at the level of hidden layers (trainable weights)[31,38]. However, training of the model is generically performed by optimizing the contracted trainable weights of each layers based on standard optimization techniques such as gradient descent and automatic differentiation. Despite being efficient and accurate, these optimization schemes only target the global minimum of a loss function while being blind to the correlations and entanglement structure between the model parameters, in addition to being hard to scale. The behavior of a loss function monitors the training convergence in these approaches, and distinguishing local minima from actual global minima is in principle very difficult. Moreover, it is also difficult to infer meaningful information from such updated weight tensors.

In this paper we resolve these issues by fully integrating NNs with TNs. More specifically, we introduce an efficient TN layer in NN structures such that the trainable weights are represented by matrix product operators (MPO)[31]. The TN layer can replace the fully connected (`Dense`) hidden layers of a NN. The resulting TNN is scalable and can have any desired number of TN layers to form a truly deep NN. We further introduce an entanglement-aware variational DMRG-like training algorithm for the resulting TNN. In contrast to previously-developed TN machine learning algorithms, which are single-layer

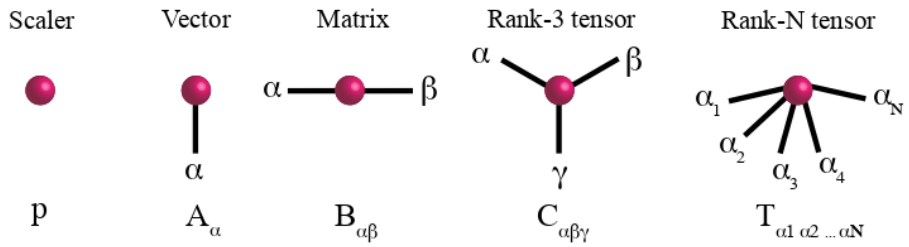


Figure 1. [Color online] Diagrammatic representation of tensors. In this notation, a rank-n tensor is represented by a solid object (here illustrated with spheres) with n connected links (legs) so that a scalar, a vector, and a matrix are objects with zero, one, and two connected links, respectively. The number of links distinguishes the number of dimensions of the tensors. For example, a vector (matrix) is a one (two) dimensional array represented by a sphere with one (two) connected link(s).

and goal-specific[30,34], our TNN is an instance of a deep neural network that can be used for different data analysis tasks such as regression and classification. The DMRG-like training algorithm provides direct access to the entanglement spectrum of the MPO trainable weights, in turn giving rise to a clear insight on the correlations in the parameters of the machine learning model. Moreover, the entanglement structure of the MPOs and their expressive power as a quantum neural state can also be assessed by typical quantum information quantities such as, e.g., the entanglement entropy of a bipartition.

In order to demonstrate the efficiency and accuracy of our technique, we use our TN layer combined with different loss and activation functions to design different deep learning models. We use such algorithms for, e.g., linear and non-linear regression, classification as well as image recognition. Next, we use our DMRG-like algorithm[41,42] to train the models. Our findings show that the TNNs have potential to be used as an alternative NN algorithm for different data analysis tasks with a general purpose. We also show how the non-linearity in the correlations amongst input data reflects itself in the entanglement spectrum and entanglement entropy of the MPO trainable weights. Our findings suggest that the DMRG-like training algorithm is an efficient numerical tool for machine learning that can potentially be used for detecting the TN representation of quantum neural states.

The paper is organized as follows: In Sec. we provide a brief overview of tensor networks and how a standard neural network composed of fully connected `Dense` layers can be tensorized to obtain a TNN. Next in Sec. 0.2 we introduce our DMRG-like training algorithm, together with the details of local tensor optimization updates based on gradient descent. In Sec. 0.8 we build TNN models for regression and use them for fitting linear- and non-linear random data. More examples of applications of TNNs for classification of labeled data are provided in Sec. 0.6. Additionally, benchmarks for image recognition of MNIST handwritten digits are presented in Sec. 0.10. Finally, in Sec. 0.10 we wrap up with our conclusions and discussions about further possible work.

Tensor Neural Networks

Tensor networks[29,43] where originally developed in physics with the aim of providing efficient representations for quantum many-body wave functions. They are also the basis of well-established numerical techniques such as density matrix renormalization group (DMRG)[41,42] and time-evolving block decimation (TEBD)[44,45]. However, due to its high potential for efficient data representation and compression, novel applications of TNs are emerging in different branches of data science such as machine learning and optimization. In this section we show how TNs can enhance deep neural networks by providing an efficient representation of the trainable weights of classical neural networks. To this end, we first review some basic concepts on TNs in order to establish basic notation and concepts widely used in the physics' context.

0.1 Tensor Network Basics

A tensor is a multi-dimensional array of complex numbers represented by $T_{\alpha\beta\gamma\ldots}$ in which the subscripts denote the tensor dimensions. The number of tensor dimensions further corresponds to the rank of the tensor. Tensors and tensor operations can alternatively be described by using tensor network diagrams[28] as illustrated in Fig. 1. Within this graphical notation, a rank-n tensor is represented by a solid object (here illustrated by spheres) with n connected links (legs) so that a scalar, a vector, and a matrix are objects with zero, one, and two connected links, respectively. This diagrammatic notation is then generalized to rank-n tensors to object with n legs, each corresponding to a tensor index.

TN diagrams not only represent the tensors but also represent tensor contractions, which is the generalization of matrix multiplication to rank-n tensors. For example, contraction of two rank-2 tensors, i.e., matrices $R_{\alpha\beta}$ and $S_{\beta\gamma}$ can be represented diagrammatically by connecting the two tensors along their shared link β, as shown in Fig. 2. This operation can further be

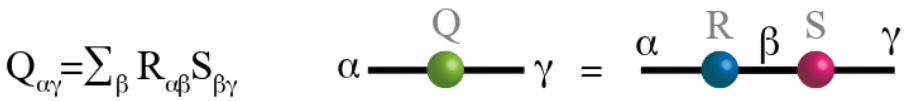


Figure 2. [Color online] Contraction of tensors, equivalent to tensor trace over their shared indices. This is represented graphically by connecting the shared links. Here, R and S tensors are connected along the shared leg β. This contraction operation is equivalent to matrix multiplication.

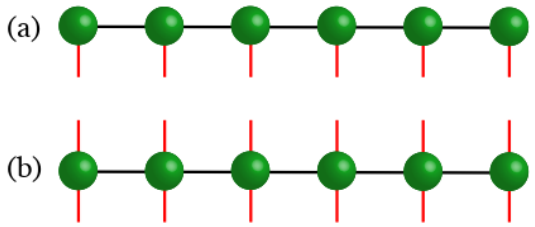


Figure 3. [Color online] Examples of well-known one dimensional TNs. (a) Matrix product states (MPS) and (b) matrix product operators (MPO). The latter will be used for representing the trainable weights of `TNLayers`.

represented mathematically as

$$Q_{\alpha\gamma} = \mathrm{tTR}(R_{\alpha\beta}S_{\beta\gamma}) = \sum_{\beta} R_{\alpha\beta}S_{\beta\gamma}, \tag{1}$$

where the tTR is the tensor trace over shared indices (tensor legs).

Large matrices and multi-dimensional arrays with a massive number of parameters can further be represented efficiently by tensor networks. Fig. 3 shows examples of two well-known tensor networks, namely, matrix product states (MPS) (also known as a tensor trains) and matrix product operators (MPO)[29], which are used for representing one-dimensional quantum many-body wave functions and operators. In particular, one can provide efficient MPO representations of large matrices by reducing the number of parameters in a controlled way. More specifically, a matrix can be decomposed as an MPO by applying singular value decomposition (SVD) and truncating the negligible singular values. An example of MPO factorization for a matrix $M_{p^2\times p^2}$ is shown in Fig. 4. Reshaping M into a rank-4 tensor and applying an appropriate SVD, one can represent matrix M as a 2-site MPO with tensors $L_{pp\chi}$ and $R_{\chi pp}$:

$$M = USV^{\dagger} = LR, \qquad L = U\sqrt{S}, \qquad R = \sqrt{S}V^{\dagger}, \tag{2}$$

where U and $V^{\dagger}$ are unitary matrices reshaped into rank-3 tensors and S is a diagonal matrix of singular values. For the $p^2 \times p^2$ matrix M there exist at most $\chi = p^2$ singular values. The non-zero singular values quantify the amount of entanglement (correlation) between the L and R MPO tensors. In a weakly correlated system, most of the singular values are close to zero and can be discarded so that only the $\chi_t \leq \chi$ largest singular values are kept. One can, therefore, reduce the number of parameters along the so-called *virtual* tensor dimensions (black link) by keeping only the most relevant degrees of freedom for the correlations between L and R. In turn, this also implies that the interconnecting tensor dimensions which emerge in the decomposition capture the relevant degrees of freedom quantifying correlations in the tensor network.

0.2 Tensorizing Standard Neural Networks

In this subsection, we show how to integrate tensor networks into standard neural networks to build a TNN. Fig. 5-(a) shows the structure of a classic NN with one fully connected `Dense` layer of hidden units (neurons). The network prediction, y_p, is

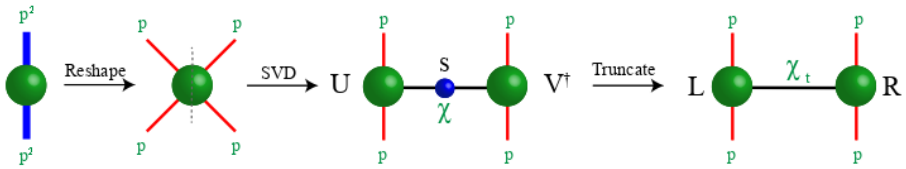


Figure 4. [Color online] Singular value decomposition of a $p^2 \times p^2$ matrix M into the L and R tensors of a 2-site MPO, see text for details.

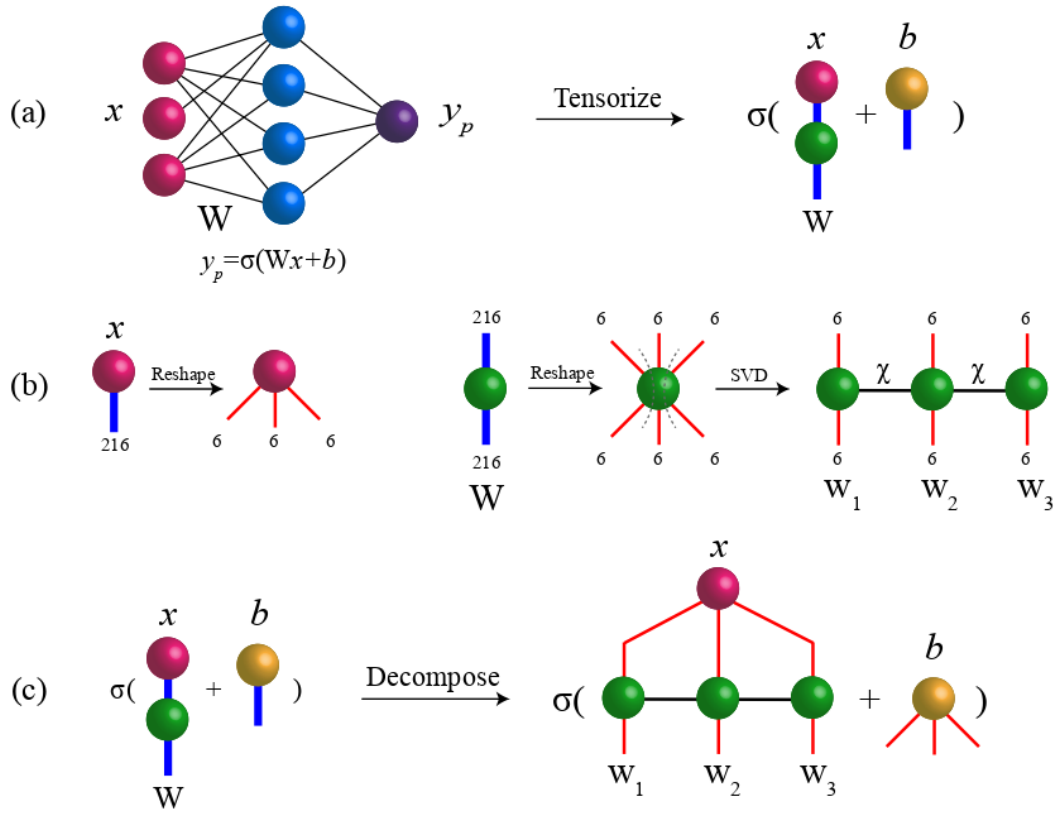


Figure 5. [Color online] Tensorizing a NN with one fully connected dense layer: (a) TN representation of the NN; (b) MPO decomposition of the weight matrix W; (c) The resulting tensor neural network with MPO trainable weights.

obtained by feeding the input feature vector x to the model as

$$y_p = \sigma(Wx+b), \tag{3}$$

where W is the weight matrix, b is some bias vector and σ is the activation function (e.g., `ReLu` or `Sigmoid`). Training the NN amounts to finding the optimum values for the parameters of the W weight matrix such that the y_p corresponds the actual label of the data.

Depending on the size of the problem, the weight matrices can be considerably large. This introduces computational bottlenecks to the NN model from the point of view of the required memory for storing such matrices, and also due to the large training time. The problem becomes more dramatic when dealing with deep NNs with many hidden layers. Optimizing the parameters of such huge weights will reduce the accuracy and efficiency of the model. It is therefore a necessity to reduce the number of model parameters, without sacrificing accuracy. To this end, we should resort to a controllable truncation scheme such that we only discard the least important information and keep the most relevant one. An example of such clever data compression schemes based on TN and MPO decomposition has already been introduced in the previous subsection. Using MPO as an efficient representation for weight matrices of a NN was originally suggested in the ML community under the name tensor trains[46] and later reintroduced in other contexts for systematic compression of fully connected NN models[36,47], for solving partial differential equations with NNs[31] and for language models[48] and speech processing[49]. Applications of tensor trains for classification and regression has also been put forward in Ref.[50]. Moreover, tensor networks have also been used for classical simulations of quantum circuits[51,52] which are also relevant for quantum machine learning and information processing.

Figure 5 further shows how the weight matrix of a fully connected `Dense` layer can be replaced by its MPO form obtained from consecutive applications of SVDs to the W matrix. The new tensorized layer, called `TNLayer`, now has several trainable weights w_i represented by MPOs. Reshaping the feature vector x to match the MPO dimensions, the network prediction y_p is obtained by contracting the resulting tensor network as shown in 5-(c). The resulting tensorized neural network is then called a TNN.

While the idea of a TNN is generic, direct tensorization can be applied to models whose feature vector x is factorable to integers, so that it could match the size and number dimensions of the MPO weights. An example of a valid-size feature vector

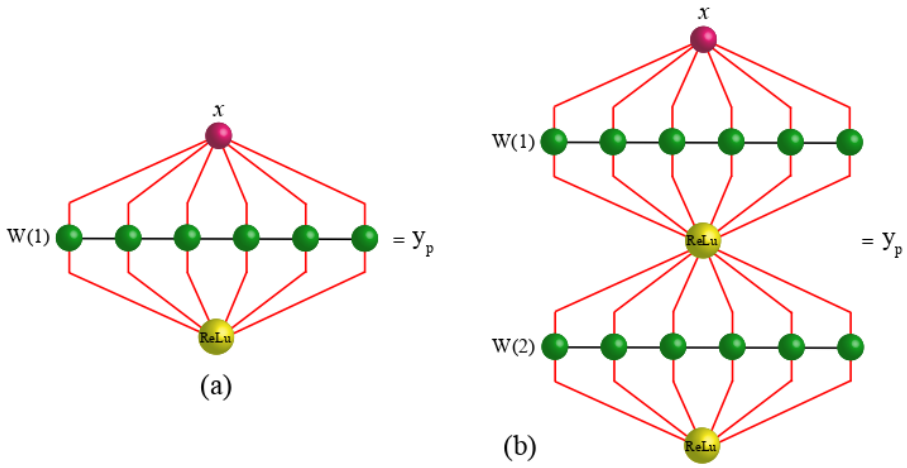


Figure 6. [Color online] Examples of fully tensorized TNNs with (a) one `TNLayer` and (b) with two `TNLayers`.

x with 216 entries is shown in Fig. 5-(b). Such a vector can be reshaped into a rank-3 tensor with dimension size 6 that is can be contracted to a `TNLayer` with three MPO weights, each with input size 6. The features that are not factorizable to match the MPOs in the `TNLayer`, can rather be transformed in the preprocessing of the machine learning task so that their new size matches the input size of the `TNLayer`. Otherwise, one can introduce a dummy non-trainable `Dense` layer with size $N_F \times N_T$ in front of the `TNLayer` to compensate for the size mismatch. Here N_F is the size of the feature vector and N_T is the input size of the contracted `TNLayer`. The non-trainable `Dense` layer can be initialized either randomly or according to a specific desired distribution. In theory, the choice of initialization for this dummy layer may slightly influence the convergence of the training algorithm and the path of the gradient toward the global minimum. In the experiments discussed in subsequent sections, we tested various random configurations and observed that their impact on the final outcomes was almost negligible.

Let us further stress that applying the activation on the TN layer is a non-linear operation and, therefore, cannot be tensorized or be applied to each MPO tensor individually. We, therefore, have to first contract the MPOs along their virtual legs to obtain a single weight matrix and then apply the activation on this matrix to obtain the network prediction, i.e.,

$$y_p = \sigma(\text{tTR}[x_{ijk\ldots} w_i w_j w_k \ldots] + b_{ijk\ldots}), \tag{4}$$

where the subscripts $ijk\ldots$ run over the dimensions of the feature tensor x. Nevertheless, we showed this operation with a single activation tensor as illustrated in Fig. 6 for examples of TNN with one and two `TNLayers`. In practice, one should first contract the features and MPOs, apply the activation function and then reshape the resulting tensor to match the inputs of the next layer. This process is continued until the whole network is contracted to obtain the prediction y_p.

The mandatory contraction of MPO weights enforced by the activation function can in principle be useful because one can connect the contracted `TNLayer` to `Dense` layers and design hybrid TNN models for different ML tasks. As exemplified in Secs. 0.8 and 0.6, we will use combinations of `TNLayer` and `Dense` layers to build different TNN models for regression and classification.

DMRG-like Algorithm for TNNs

In the previous section we introduced the tensor neural networks and how to build them from `TNLayers`. In this section, we provide details about the DMRG-like training algorithm for multi-layer TNN models. Similar to generic optimizers that can be found in any ML library such as `TensorFlow`[53] or `PyTorch`[54], our DMRG-like algorithm is also of generic purpose and can be used for training different models ranging from regression to classification.

0.3 Local gradient-descent update

The generic idea of training a neural network is to find the optimum values for the weight matrices of the NN layers by minimizing a loss function. In a feed-forward NN, all the data or batches of it are fed to the network to calculate the y_p in the forward path. Next, the gradients of trainable weights with respect to the loss function are calculated in the backward path, and the weights are updated accordingly with a gradient-descent (GD) step. This whole process is iterated until a convergence criterion is met. Once trained, the model is tested to predict on unseen test data. The performance of the model is further evaluated by measuring different accuracy metrics.

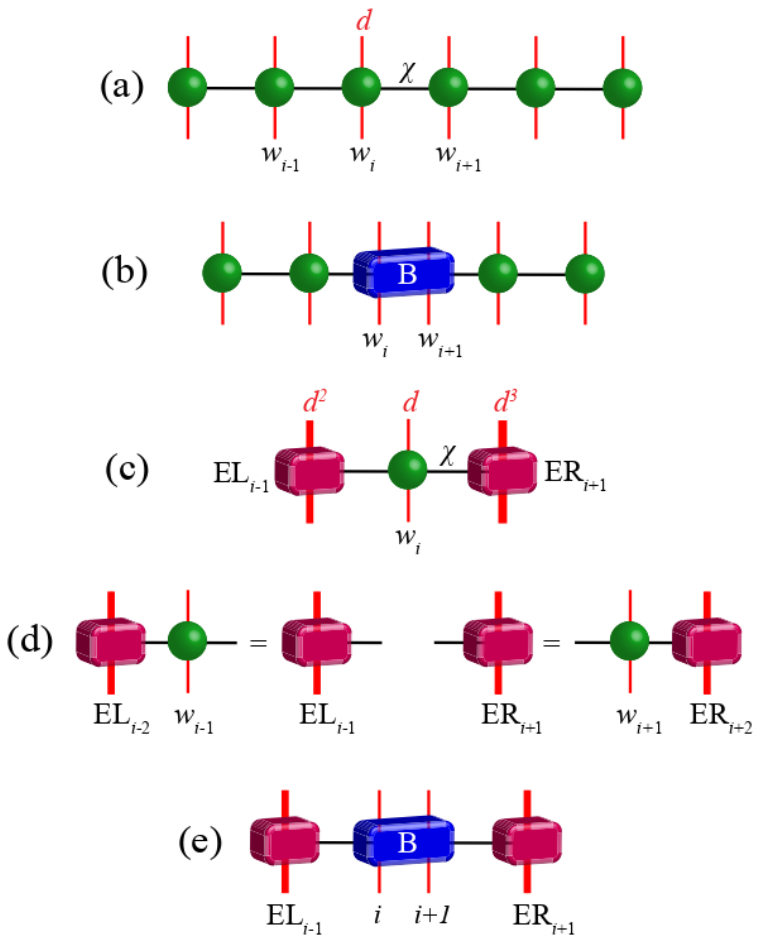


Figure 7. [Color online] Defining bond and environment tensors: (a) MPO factorized weights; (b) Bond tensor $B_{i,i+1}$ (shown in blue) which is obtained from contracting a pair of neighboring MPO tensors, w_i, w_{i+1}; (c) Left and right environment tensors of each w_i MPO tensor; (d) A single step of obtaining environment tensors; (e) Alternative representation of MPO weights in terms of bond tensor $B_{i,i+1}$ and its surrounding environment.

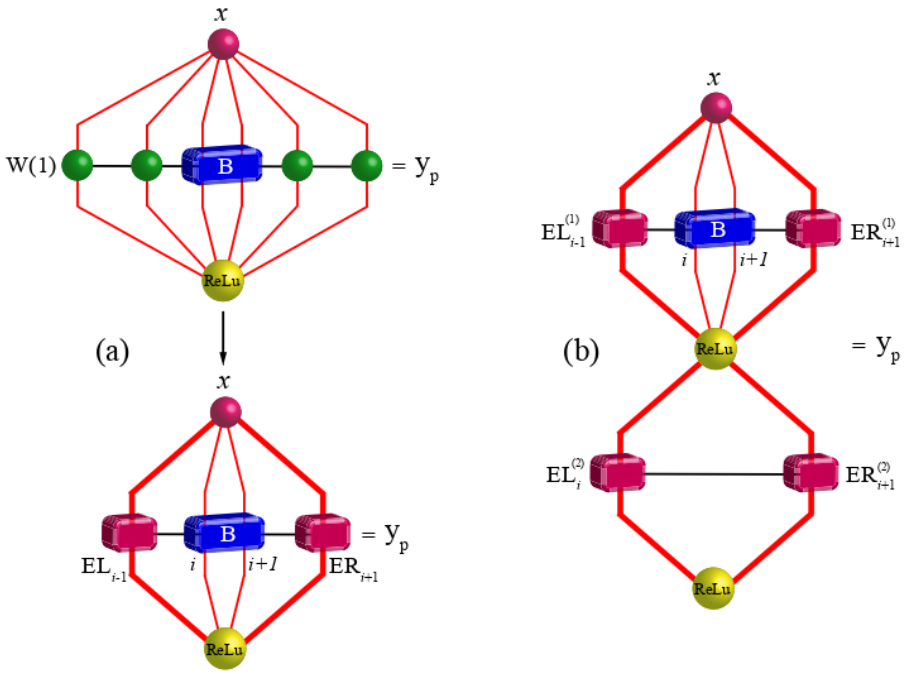


Figure 8. [Color online] Alternative representation of a TNN in terms of bond tensor $B_{i,i+1}$ and environments for (a) a TNN with one `TNLayer` and (b) two `TNLayers`.

In contrast to classic NNs with `Dense` layers, the TNN is composed of `TNLayers` that have multiple trainable MPO tensors. While a similar GD approach can be used for training the MPOs[31], here we use a DMRG-like technique to update the MPO tensors by pairs, with a sweeping local gradient-descent (LGD) algorithm. However, before we present the detail of the update, the following remarks are in order: i) Given a pair of neighboring MPO tensors, w_i, w_{i+1}, a *bond* tensor $B_{i,i+1}$ is obtained by contracting the w_i and w_{i+1} along their shared virtual dimension (see Fig. 7-(a,b)). ii) For every MPO tensor w_i, the contraction of all tensors at the right side and left side of w_i are called environment. These are denoted by EL_{i-1}, ER_{i+1} for the left and right environments, respectively (see Fig. 7-(c,d)). iii) For every bond tensor $B_{i,i+1}$, the `TNLayer` and further the overall TNN can be represented in terms of the bond tensor and its left and right environments, as shown in Fig. 7-(e) and Fig. 8. Given the fact that the TNN has to be contracted for every $B_{i,i+1}$ pair, which is required for updating the MPO weights, introducing the environment tensor can substantially reduce the computational cost of the network contraction.

Having the aforementioned remarks in mind, we train the TNN by sweeping over the MPO pairs of tensors of each `TNLayer` as follows:

1. Do for all MPO tensor pairs $\{w_i, w_{i+1}\}$ where $i \in [1, N_{\text{MPO}} - 1]$ (left to right sweep):

   (a) Build the local bond tensor $B_{i,i+1}$ by contracting w_i and w_{i+1}.

   (b) Calculate the gradient of $B_{i,i+1}$, i.e., $\Delta B_{i,i+1}$ with respect to a loss function (see Sec. 0.4, 0.5).

   (c) Update the bond tensor with learning rate α: $B'_{i,i+1} = B_{i,i+1} - \alpha \Delta B_{i,i+1}$.

   (d) Split the updated bond tensor by SVD: $B'_{i,i+1} = USV^\dagger$.

   (e) Truncate the matrices to keep the desired number of singular values: $B'_{i,i+1} \approx U'S'V'^\dagger$.

   (f) Update the weights: $w'_i = U'\sqrt{S'}$ and $w'_{i+1} = \sqrt{S'}V'^\dagger$.

   (g) Update the left and right environment tensors: $EL'_i = EL_{i-1}w'_i$ and $ER'_{i+1} = w'_{i+1}ER_{i+2}$.

2. Sweep back from right to left and repeat the same pair update.

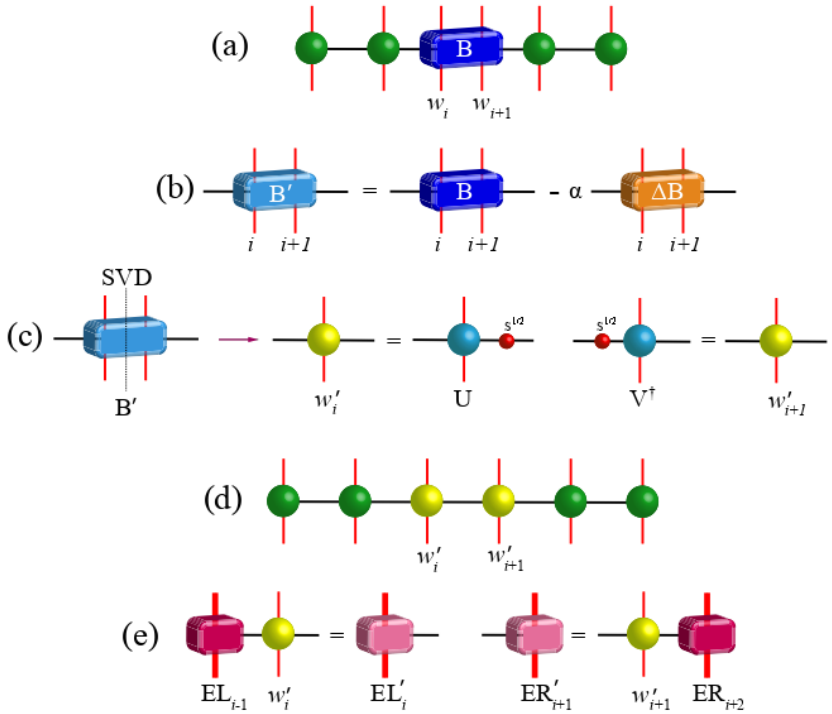


Figure 9. [Color online] Graphical representation of the DMRG-like sweeping algorithm for training TNNs: (a) MPO weights with bond tensor $B_{i,i+1}$; (b) Updating the bond tensor with LGD step; (c) Calculating the updated local weights w_i' and w_{i+1}' from the updated bond tensor $B_{i,i+1}'$ by SVD and truncation; (e) Updating the left and right environments.

Details about the LGD algorithm in terms of TN diagrams are also provided in Fig. 9.

Let us further note that other instances of DMRG-based training algorithms have already been introduced in previous studies such as in Ref.[46] that the author uses a one-site sweeping algorithm for training the MPO layers or the single-layer DMRG sweeping algorithm of Ref.[30] for kernel learning. Our training algorithm differs from previous approaches in the sense that first, it is based on two-site DMRG sweeping update, which is known from simulation of quantum many-body systems to be more stable. The two site update, as will be discussed in future sections, can provide access to the entanglement structure of the weight tensors and shed light on the expressive power of TNNs. Second, in contrast to Ref.[30], our sweeping algorithm is not limited to only a single layer and can be straightforwardly adapted to multilayer and deep networks with many layers. Moreover, our training algorithm allows designing TNN models with mixed `Dense` and `TNLayer`s, bringing huge flexibility to perform generic ML tasks.

0.4 Tensor gradient for linear activations

The DMRG-like sweeping algorithm for training the TNN is a gradient-based optimization approach in which the local bond tensors, $B_{i,i+1}$, are updated towards a global minimum of a loss function by a gradient descent step with learning rate α. Gradient of the bond tensors with respect to the loss, $\Delta B_{i,i+1}$, are therefore required for updating the weights. Given the fact that the TNN is a tensor network, gradients of the tensors in such a network with respect to a desired loss function are simply the network itself, with the corresponding tensor removed from the network. More specifically, consider a desired loss function such as the mean-squared error (MSE),

$$L_{\text{MSE}} = \frac{1}{N_s}\sum_{j=0}^{N_s}(y_p - y_j)^2, \tag{5}$$

where N_s is the number of training samples (input features) and y_js are their labels. Defining $y_p = MB$, where M is the contraction of all tensors in the TNN excluding the bond tensor B (see Fig. 10), the L_{MSE} alternatively reads

$$L_{\text{MSE}} = \frac{1}{N_s}\sum_{j=0}^{N_s}(MB - y_j)^2. \tag{6}$$

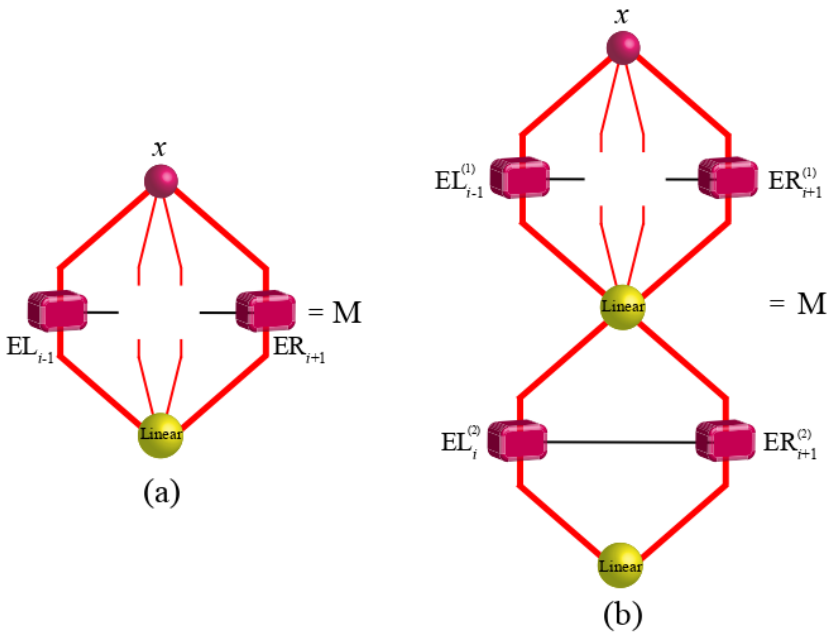


Figure 10. [Color online] The M tensor required for manual calculation of the gradient $\Delta B_{i,i+1}$ for a TNN with (a) a single `TNLayer`, and (b) two `TNLayers`, and with linear activation functions.

Taking the derivative with respect to B, the gradient of local bond tensors is given by

$$\Delta B = -\frac{\partial L_{\text{MSE}}}{\partial B} = \frac{2}{N_s}\sum_{n=1}^{N_s}(y_j - y_p)M. \tag{7}$$

Note that while both the M tensor and y_p involve the contraction of the TNN, we do not need to do it twice. In practice, we first calculate the M tensor and then we contract it to the bond tensor B to obtain the y_p prediction.

The above procedure for manual calculation of the gradient is only valid for linear activations or any other activation function that can be applied to the individual MPO weight tensors and not the contracted network. The reason is that the M tensor in Fig. 10 is obtained by removing the bond tensor B from the whole network after the application of the activation function. It has already been pointed out that for non-linear activations, one has to first contract the MPOs and the input from the previous layer and then apply the activation. It is obvious that the B tensor can not be removed from a contracted network. Therefore, no M tensor can be formed to calculate the tensor gradients manually.

Let us further remark that while the gradient in Eq. (7) and, subsequently, the LGD update (introduced in the previous subsection) are local, the DMRG-like sweep will restore the global features and correlations once iterated sufficiently.

0.5 Automatic gradient

Although the previous tensor gradient approach is suitable for TNN models fully composed of `TNLayers` and linear activation functions, it will not be efficient and flexible once dealing with models with hybrid architectures containing a mixture of `Dense` and `TNLayers` and nonlinear activation functions. Given the fact that our TNN is a feed-forward neural network, we can use automatic differentiation schemes and obtain the gradient of the TNN trainable weights with back-propagation (see Ref.[55,56] for a review on automatic differentiation and back-propagation).

While one can implement the whole process manually, implementing the `TNLayer` in one of the ML libraries that support the automatic differentiation is highly recommended. Here we used the `TensorFlow` library and one of its useful features, i.e., `GradientTape`[53] to record all the mathematical operations in the forward path. These include the contraction of all tensors and calculation of the network prediction, y_p, as well as the loss, ex. Eq.(5). The `GradientTape` uses the operation records of the forward path and calculates the gradient of all trainable vectors, matrices and tensors in the backward move. Once the gradients are obtained, every trainable weights of the TNN can be updated. The MPOs are updated as prescribed in steps (c)-(g) of the DMRG-like algorithm of Sec. 0.3. The bias vectors, b, and the weight matrices of the `Dense` layers W can further be updated as

$$W' = W - \alpha\Delta W, \qquad b' = b - \alpha\Delta b. \tag{8}$$

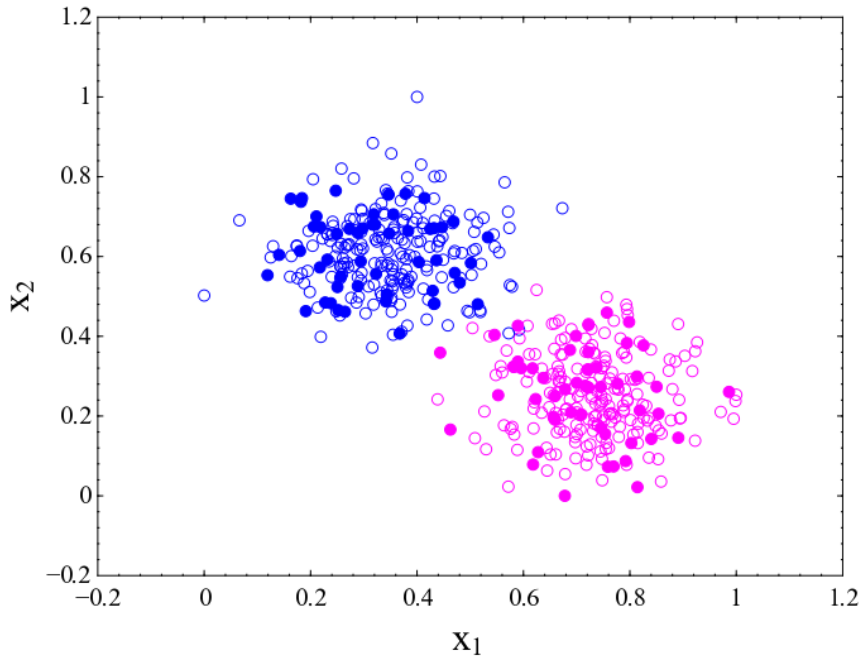


Figure 11. [Color online] Binary classification of isotropic Gaussian blobs with TNN 1. The empty (filled) circles represent training (test) data.

For hybrid TNN models composed of multiple `TNLayers` and `Dense` layers one can in principle update the weights with different strategies, i.e., layer-by-layer (LbL) or partial-all-layer (PaL). In the LbL approach, we start updating from the first layer until we reach the last output layer. The `Dense` layers are updated according to Eq. (8) and the MPO weights of `TNLayers` are updated by a few sweeps say $N_{\text{sweep}} \leq 10$ according to the DMRG-like algorithm of Sec. 0.3. On the other hand, the PaL approach targets all the layers at once. In this scheme, we consider a large $N_{\text{sweep}} \geq 2000$ and update the MPO pairs $\{w_i^l, w_{i+1}^l\}$ of all `TNLayers` (l is the layer index), i.e, we do a multi-layer sweep. With each MPO pair update, we update the dense layers as well. This will help to reflect the local update of MPO weights in the matrix weights of the `Dense` layers during the sweep. The multi-layer sweep is performed until the loss converges to a certain threshold or until the N_{sweep} is reached.

It is also worth noting that for training the example models that we provide in the next sections for regression and classification, we observed that the PaL approach with large sweeps works much better from the point of view of convergence and accuracy. However the number of sweeps also depends on the dataset as well. For example for the case of MNIST handwritten digits since the number of record in the datasets are very large, even a single sweep is sufficient to obtain a high accuracy.

0.6 Entanglement Entropy

In classic NN algorithms, the training convergence is checked by monitoring the loss function or some accuracy metric. The main drawback of these approaches is that they do not provide any information about the nature of the weights, the correlation among their parameters, and their expressive power. Thanks to the DMRG-like update, we have access to the singular values at every step of the sweep iterations, and this provides valuable information about the correlation (entanglement in quantum case) between MPO tensors. The individual singular values along the virtual dimensions of MPO tensors or their accumulative behavior in terms of the entropy, $S = \sum_i \lambda_i^2 \log \lambda_i^2$, reveal the degree of correlation (entanglement) between pieces of trainable weights. While weakly correlated states are distinguished by zero or very small entanglement entropy with only a few non-zero singular values, entangled states will have non-vanishing S and many non-zero singular values. Moreover, observing the behavior of entanglement entropy and singular values during the training can be used as another convergence measure, on top of the loss, which directly sheds light on the convergence of the MPO parameters.

Let us further point out that in each `TNLayer` with N_{MPO} tensors, we have $N_{\text{MPO}} - 1$ virtual bonds. We can calculate the entanglement entropy on each of these bonds. The entanglement entropy values may vary based on the model's architecture, the input data, and the size of the virtual dimensions. Therefore, one can examine the individual or average properties of these entanglement entropies to assess the convergence or correlations among the model parameters.

Table 1. TNN architecture for binary classification of random Gaussian blobs

$N_s = 500$, Sweep = 2000, $\alpha = 0.1$, Batch = 500
$\mathscr{I} = \left\{ \{x_1, x_2\}, \{y\} \middle\| 0 \le x_1, x_2 \le 1,\ y = [0,1] \right\}$
$\downarrow$ { `Dense`{Units $= 64$, trainable = `False`} Activation: `None`
$\downarrow$ { `TNLayer`{Units $= 64, N_{\mathrm{MPO}} = 6$} Activation: `ReLU`
$\downarrow$ { `Dense`{Units $= 2$} Activation: `Softmax`
$\mathscr{O} = [0,1]$
Loss Function: `BinaryCrossEntropy`

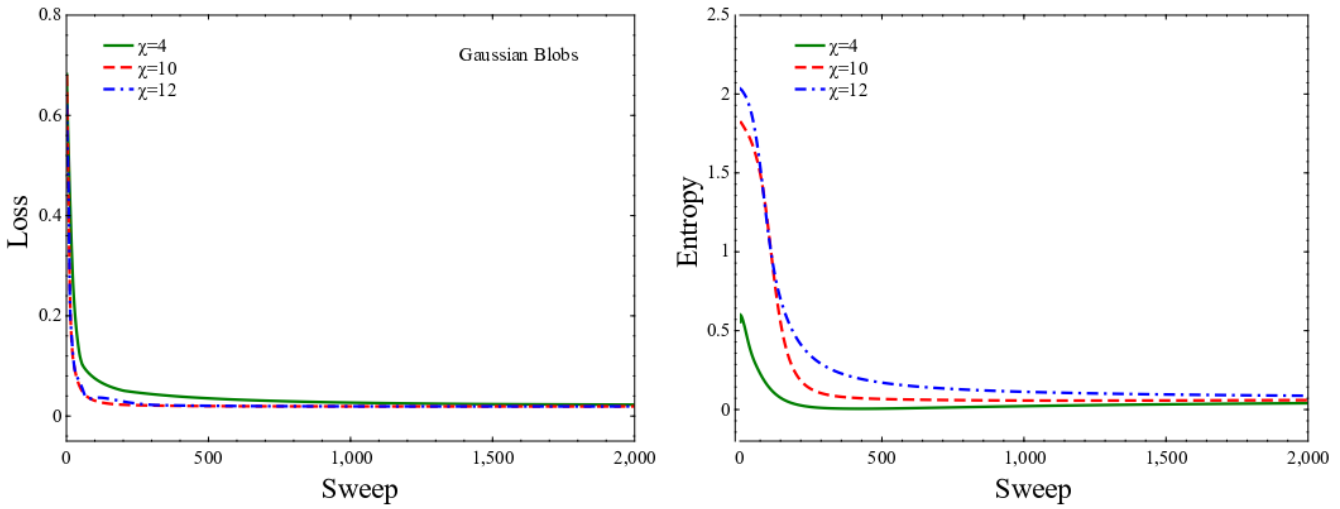


Figure 12. [Color online] (left) Training loss and (right) the S entanglement entropy on the middle virtual bond of the MPO weights as a function of DMRG-like sweeps for training the TNN model 1 used for classification of Gaussian blobs.

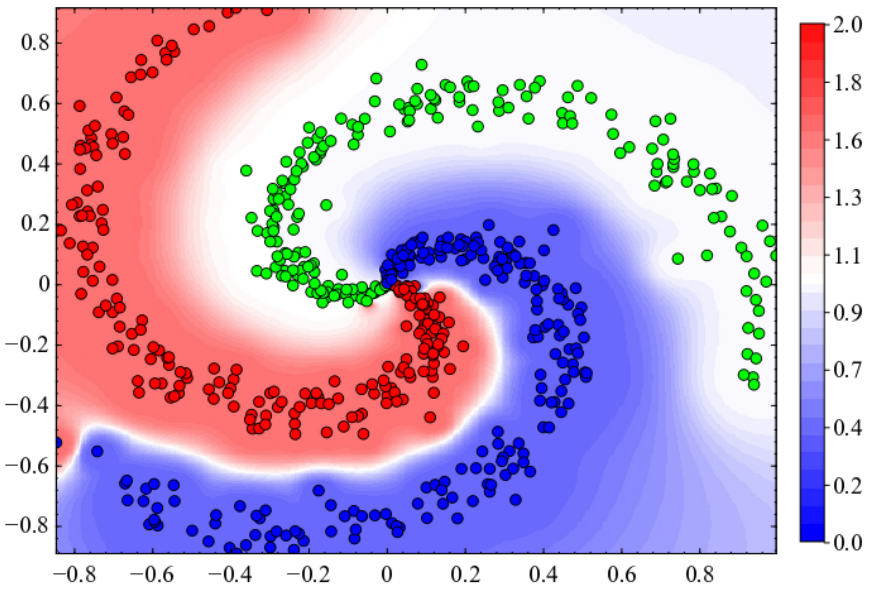


Figure 13. [Color online] Ternary classification of random spiral distribution with three classes. The red, blue and green filled circles distinguish the training data in each class. The red, white and blue shaded regions further distinguish the decision boundary obtained from training the TNN model 2.

TNN Classifier

After introducing the TNN and the DMRG-like sweeping algorithm, let us see the performance and accuracy of the technique for classification tasks. In what follows, we present two examples for the classification of labeled data, one for the isotropic Gaussian blobs and another for the spiral distribution, and test the TNN in action.

0.7 Gaussian Blobs

As first example, we use the TNN to model a binary classification and train it over the random isotropic Gaussian blobs as shown with empty circles in Fig. 11. Each sample in the dataset has two position features denoted by x_1, x_2 and a label $y = [0, 1]$. The architecture of the TNN model used for the binary classification of the Gaussian blobs is detailed in Table 1. The model is composed of a `TNLayer` and two `Dense` layers. The first `Dense` is non-trainable and has only been added to compensate for the size mismatch between the features and the next `TNLayer` (see Sec. 0.2). This dummy layer has been initialized with random values. The `TNLayer` has six MPO trainable weights, each with input and output dimensions $d = 2$, and virtual dimension χ, and `ReLu` activation function. Finally the output layer is a `Dense` layer with `Softmax` activation which delivers the predicted probabilities of both labels as one-hot-encoded (OHE) vectors. The prediction y_p can then be read form the `argmax` of the OHE probabilities for each sample.

Fig. 11 shows the distribution of the two Gaussian blobs clusters. The empty circles denote the training data and the filled circles represent the test data. We performed the training for virtual dimension $\chi \in [2, 12]$ over 500 training data and tested the model over 100 unseen data. The average accuracy for classifying Gaussian blobs with the TNN model 1 is 99% (see Table 1 for training hyper-parameters).

Fig. 12 further shows the loss function and entanglement entropy on the middle virtual bond of the MPO weights, as a function of the sweep iterations for different bond dimensions χ. While both plots provide a measure for the training convergence, the S suggests that the MPO weights of the `TNLayer` represent a weekly entangled state with a small correlation among the parameters. This is best confirmed by very small entanglement entropy (close to zero) which is a typical behavior for product states. Looking at the distribution of blobs that are categorized into two localized clusters with almost no overlap, it is expected of the MPO weights be very close to that of the product states. The TNN therefore, not only classifies the blobs but also provides expressive insight into the nature of MPO weights.

0.8 Spiral Distribution

For the second example, we consider a random spiral distribution with three classes as illustrated in Fig. 13. We use the TNN model described in Table 2 for the ternary classification of spiral data. The model is similar to the previous example except

Table 2. TNN architecture for the classification of random spiral distributions

$N_s = 600$, Sweep = 3000, $\alpha = 0.1$, Batch = 600
$\mathscr{I} = \left\{ \{x_1, x_2\}, \{y\} \middle\| -1 \le x_1, x_2 \le 1,\ y = [0,1,2] \right\}$
$\downarrow$ { `Dense`{Units = 64, trainable = `False`} Activation: `None`
$\downarrow$ { `TNLayer`{Units = 64, $N_{\text{MPO}} = 6$} Activation: `ReLU`
$\downarrow$ { `Dense`{Units = 3} Activation: `Softmax`
$\mathscr{O}$ = [0,1,2]
Loss Function: `BinaryCrossEntropy`

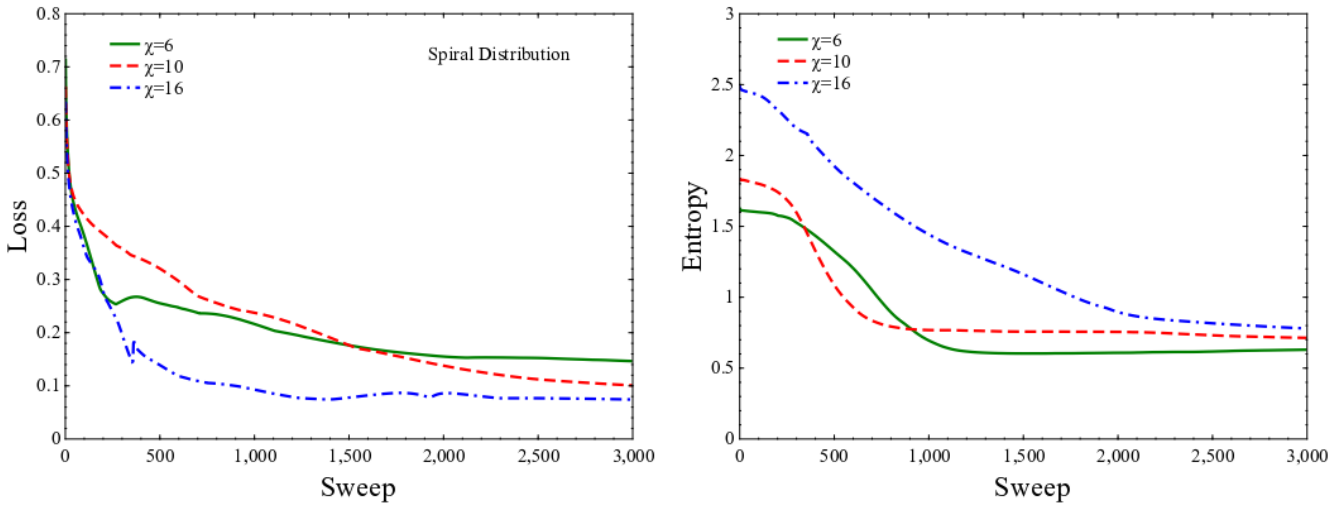


Figure 14. [Color online] (left) Training loss and (right) the S entanglement entropy on the middle virtual bond of the MPO weights as a function of DMRG-like sweeps for training the TNN model 2 used for classification of spiral distibution.

that now the output `Dense` layer has three outputs for the probabilities of each class. We trained the model over 600 samples, 200, for each class and obtained the decision boundary of the spiral distribution as depicted in the shaded regions with red, white, and blue colors in Fig. 13. The average accuracy of the model from different runs is 95% (see Table 2 for training hyper-parameters).

The loss function and entanglement entropy, on the middle virtual bond of the MPO weights, during the training have also been displayed in Fig. 14 for different bond dimensions. Compared to gaussian blobs, one can clearly see that the behavior of loss and entanglement is totally different for the spiral distribution. Looking at the distribution of points in Fig. 13, the spiral dataset is expected to be more correlated and therefore, more challenging to be trained. The behavior of the loss as well as the non-vanishing entanglement entropy indeed confirms the higher degree of entanglement among the parameters of the MPOs for the spiral distribution. The S for the spiral distribution is approaching ≈ 0.75 signaling an entangled structure with more correlated parameters than that of the Gaussian blobs.

TNN Regressor

In this section, we challenge our TNN for regression tasks. Here we present two regression examples one for fitting a line to a random linear distribution and another one for fitting to the random points around the nonlinear sine function.

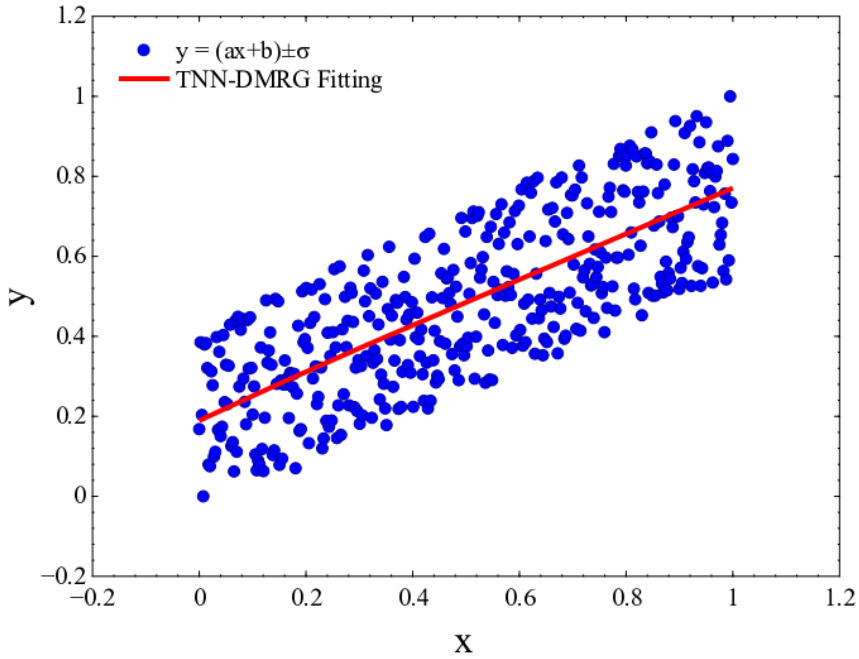


Figure 15. [Color online] Linear regression of the random data around the line $y = ax + b$ with $a = 0.55$ and $b = 0.20$. The red line is the fit obtained from training the TNN model 3. The predicted slope and shift of the red fitted line are $a = 0.57$ and $b = 0.19$.

Table 3. TNN architecture for linear regression

$N_s = 400$, Sweep = 2000, $\alpha = 0.1$, Batch = 400
$\mathscr{I} = \left\{ \{x\}, \{y\} \middle\| 0 \le x, y \le 1 \right\}$
$\downarrow$ { `Dense`{Units $= 64$, trainable = `False`}
Activation: `None`
$\downarrow$ { `TNLayer`{Units $= 64, N_{\mathrm{MPO}} = 6$}
Activation: `ReLU`
$\downarrow$ { `Dense`{Units $= 1$}
Activation: `None`
$\mathscr{O} = y_p$
Loss Function: `MeanSquareError`

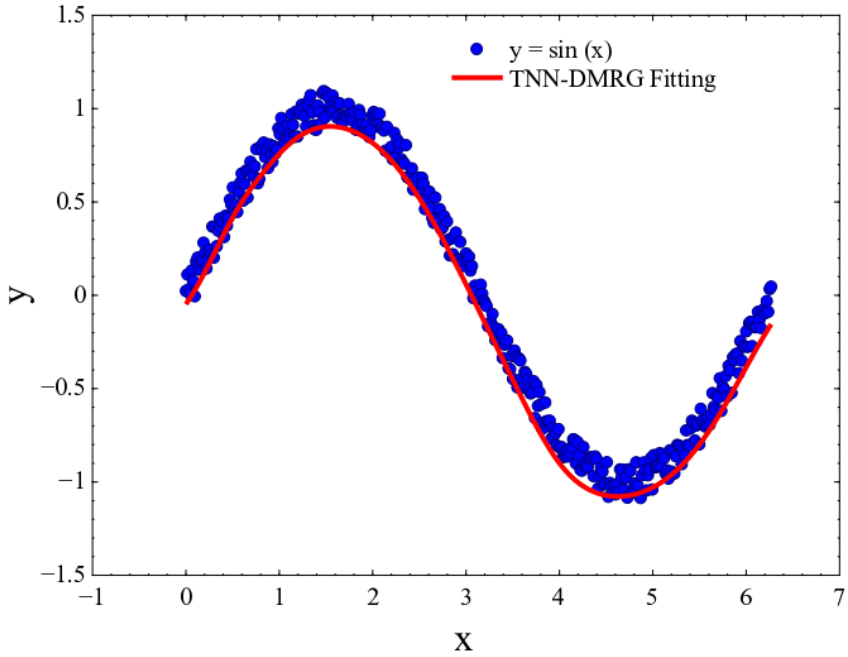


Figure 16. [Color online] Non-Linear regression of the random data introduced to the function $y = \sin x$ for $0 \leq x \leq 2\pi$. The red line is the fit obtained from training the TNN model 4.

0.9 Linear Regression

In order to test the TNN for a linear regression problem, we generate our data by adding uniform random noise to the points out of linear function $y = ax + b$ with $(a = 0.55, b = 0.20)$for $0 \leq x \leq 1$. The distribution of the linear random points has been depicted with blue circles in Fig. 15. In order to fit a line to the points, we designed a linear TTN model as described in Table 3. The model is composed of a single trainable `TNLayer` with `ReLu` activation and a `Dense` output layer with no activation. As usual, we put a random non-trainable `Dense` layer in front of the `TNLayer` to compensate for the shape mismatch between the features and the `TNLayer`.

Training the linear TNN model for 400 random points and 2000 sweeps, we obtain a linear line that is perfectly fitted to the random data as shown in red in Fig. 15. Reading the slope and data shift from the predicted fitted line, we obtain $a_p = 0.57$ and $b_p = 0.19$ which is in good agreement with the original a and b parameters.

0.10 Non-linearRegression

Lastly, we challenge our TNN and DMRG-like algorithm for a non-linear regression task to predict a fit to the non-linear random points obtained by uniform noise to the $y = \sin x$ function for $0 \leq x \leq 2\pi$. To this end, we introduce a non-linear TNN model with three `TNLayers` as detailed in Table 4. Neglecting the initial dummy `Dense` layer, the first `TNLayer` has a `Linear` activation, and the two others have `Sigmoid` activation functions. Adding to this structure an output `Dense` layer with `ReLu` activation, the resulting TNN model is fully capable of capturing non-linear correlation of the input features.

Fig. 16 shows 400 non-linear random *sine* shape points that have been fitted perfectly by the predicted red line from the trained TNN model. The TNN architecture of model 4 is an example of a multi-layer deep TNN which is a mixture of both `Dense` and `TNLayers`. Indeed other complicated architectures with more layers and different activations can be designed for generic ML tasks based on neural networks. The choice of these examples and hyper-parameters was to showcase the performance, accuracy, and flexibility of designing generic models with the TNN and DMRG sweeping algorithm. Indeed by changing the architecture or hyper-parameter tuning of the aforementioned models, one can improve the quality of the results. However, this was not the purpose of this study.

TNN Image Recognition

Next, to evaluate our algorithm's effectiveness in a more complex scenario, we applied the TNN model and DMRG-like sweeping algorithm to image recognition using the MNIST dataset of handwritten digits. We compared the results to those

Table 4. TNN architecture for non-linear regression

$N_s = 400$, Sweep = 2000, $\alpha = 0.1$, Batch = 400
$\mathscr{I} = \left\{ \{x\}, \{y\} \middle\| 0 \le x \le 2\pi,\ -1 \le y \le 1 \right\}$
$\downarrow$ { `Dense`{Units $= 64$, trainable = `False`} Activation: `None`
$\downarrow$ { `TNLayer`{Units $= 64, N_{\mathrm{MPO}} = 6$} Activation: `Linear`
$\downarrow$ { `TNLayer`{Units $= 64, N_{\mathrm{MPO}} = 6$} Activation: `Sigmoid`
$\downarrow$ { `TNLayer`{Units $= 64, N_{\mathrm{MPO}} = 6$} Activation: `Sigmoid`
$\downarrow$ { `Dense`{Units $= 1$} Activation: `ReLu`
$\mathscr{O} = y_p$
Loss Function: `MeanSquareError`

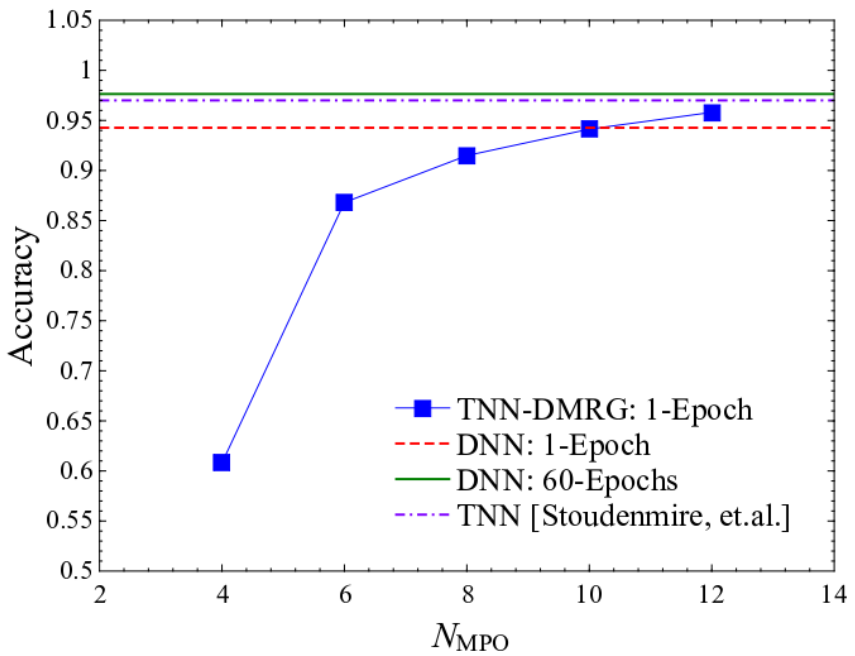


Figure 17. [Color online] Scaling of the MNIST classification accuracy with the number of tensors, N_{MPO}, in the `TNLayer` of the TNN model for one training epoch. The red-dashed and solid-green lines represent the accuracy of the classic NN model for 1 and 60 epochs (full convergence), respectively. The violet dashed-dot line further shows the accuracy of the TNN approach from Ref.[30].

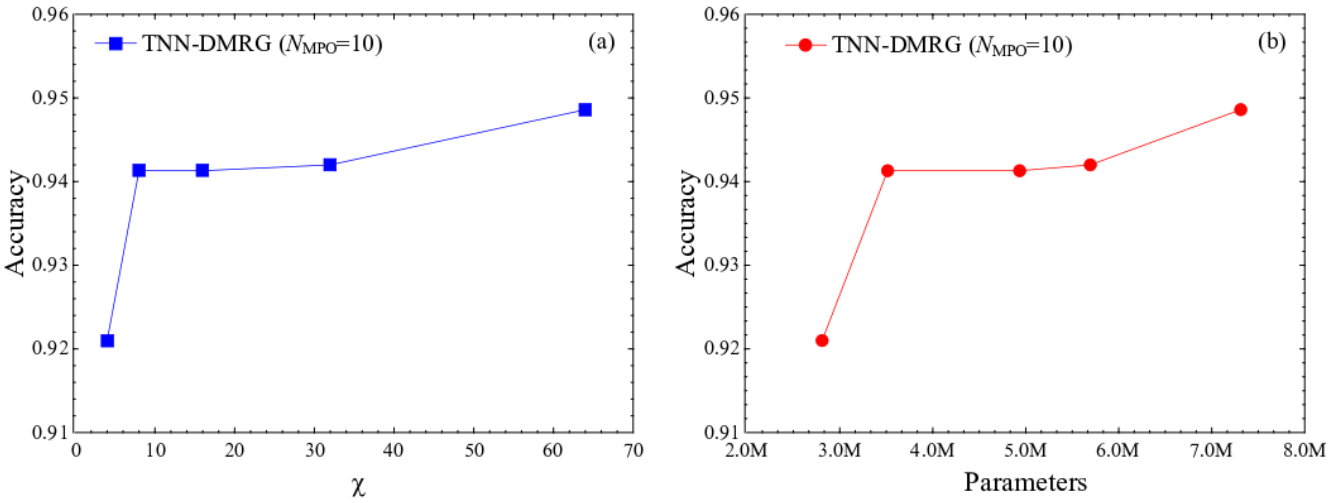


Figure 18. [Color online] Scaling of the MNIST classification accuracy (a) versus MPO virtual dimension *chi* in the `TNLayer` and (b) versus the total number of trainable parameters of the TNN model, for one training epoch. The accuracies were obtained for fixed number of tensors, i.e, $N_{\text{MPO}} = 10$.

of a classical neural network (NN) model. To this end, we designed a TNN model featuring a single `TNLayer` with `Relu` activation followed by an output `Dense` layer equipped with 10 units to predict the probabilities of the digits zero through nine. As usual, we introduced a dummy `Dense` layer with random configuration in front of the `TNLayer` to compensate for the shape mismatch with the input data. In contrast, the classical NN model employed a similar architecture but substituted the `TNLayer` with a standard `Dense` layer and omitted the dummy input layer.

While we used the DMRG-like algorithm for training the TNN model with learning rate $\alpha = 0.60$, the classical NN model was trained using the built-in `Adam` optimizer of the `TensorFlow` library with learning rate $\alpha = 0.001$. We used `BinaryCrossentropy` loss function for both models and benchmarked the accuracy of the two models in different scenarios. The MNIST dataset contains 60,000 (10,000) training (test) dataset that we used all of them for training and testing the accuracy of both models. Besides, we trained the models on a core-*i*9 MacBook Pro 2019 with six 2.9 GHz CPUs. This is relevant later where we benchmark the training time of both models.

Fig. 17 illustrates the test accuracy of the TNN model as a function of the number of tensors,N_{MPO}, in the `TNLayer`. The graph demonstrates that the accuracy in recognizing handwritten digits improves with an increase in the number of MPO tensors. The accuracies were recorded after one epoch and one sweep. Among these measurements, the highest accuracy observed was 0.958 at $N_{\text{MPO}} = 12$. In contrast, the accuracy of the corresponding classical NN model was 0.942 after one epoch, as indicated by the red-dashed line in the figure. For the reference, the accuracy of the TNN model from Ref.[30] has also been added to the plot with violet dashed-dot line. One should note that the optimization landscapes and the gradient descent optimizers used for classical NNs and TNNs differ significantly. The DMRG training algorithm searches for the global minimum in the parameter space of multiple tensors in the `TNLayer`, whereas the `ADAM` optimizer, used for the NN model, searches within the parameter spaces of a single weight matrix in the `Dense` layer of the corresponding NN. For this reason, and to evaluate the accuracy of both models after utilizing all the training data only once, we chose a single epoch as the baseline for our experiment. However, the accuracy can be enhanced by increasing the number of epochs. For example the green-solid line in Fig. 17 depict the accuracy of the NN model after 60 epochs, which reached 0.976.

Next, we benchmarked the accuracy of the TNN model on the MNIST dataset with respect to the virtual bond dimension *chi* as shown in Fig. 18-(a) for a fixed $N_{\text{MPO}} = 10$ and one training epoch. It is notable that the accuracy improves with an increase in the bond dimension. One should note that by increasing the bond dimension χ, the number of parameters in the MPO tensors of the `TNLayer` also increases. Therefore, one can alternatively present the accuracy as a function of the parameters (see Fig. 18-(b)). This behavior is expected, as increasing the number of trainable parameters can positively impact accuracy, albeit at the cost of higher computational expense.

Finally, we have benchmarked the training time of the TNN model versus the classic NN model over one training epoch. Fig. 19 depicts the scaling of the training time for 60,000 records in the MNIST dataset as a function of N_{MPO} for one sweep. While the training time for the classic NN model, using the `Adam` optimizer, is approximately 10 seconds, the training time for the TNN model, which employs the sweeping DMRG-like algorithm, is on the order of minutes and increases exponentially

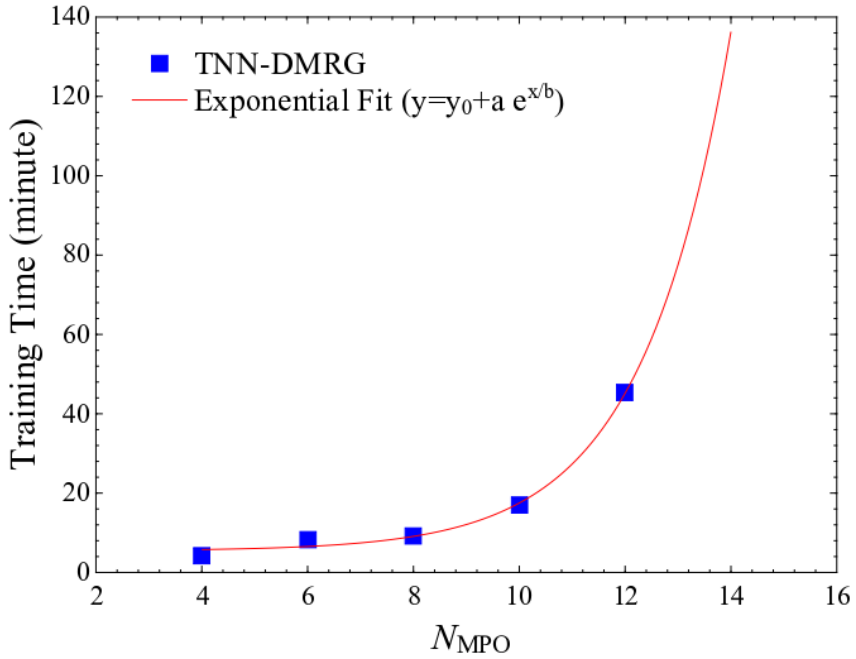


Figure 19. [Color online] Scaling of the MNIST classification training time for one epoch for with respect to the number of tensors, N_{MPO}, in the `TNLayer` of the TNN model for one training epoch. The red line is the exponential fit to the training times with $y_0 = 5.42$, $a = 0.03$ and $b = 1.67$. The training time of the corresponding classical NN is 10 seconds.

with the number of tensors, N_{MPO}. The red solid line in the figure represents an exponential fit to the TNN training times, indicating a sharp increase as the number of tensors grows.

We conclude this section by noting that the sweeping DMRG-like training algorithm presented here served as a proof of concept and has not been optimized for performance during our benchmarks. Therefore, the TNN-DMRG, in its current CPU implementation, cannot surpass conventional gradient-based optimizers such as Adam. However, there is significant room for improvement, including efficient implementations of the tensor contractions on GPU or redesigning the building blocks of the optimizer. Additionally, the TNN models can be further optimized using any of the built-in optimizers in `TensorFlow` (see Ref.[31] for an example).

Conclusions and outlook

In this paper, we introduced a fully tensorized neural network model for deep learning. By replacing the trainable weight matrices of the fully connected dense layers of classic NNs with matrix product operators, we obtained tensor neural networks that are capable of modelling different machine learning tasks ranging from classification to regression. We further introduced a new entanglement-aware training algorithm based on DMRG and local gradient-descent updates for training the TNN models which act on a reduced parameter subspace obtained from the tensorization of trainable weights. Our TNNs are generic-purpose, i.e., they can be used for automatizing different ML models such as regression and classification. Our implementation further allows to construct hybrid architectures with a mixture of `Dense` and `TNLayers` to build real instances of deep learning models.

In order to show the performance and accuracy of the TNNs and DMRG-like training algorithm, we considered several deep learning models for linear and non-linear regression, classification of labeled data as well as image recognition with MNIST handwritten digits. Our findings suggest that TNNs and DMRG-like training algorithms can serve as potential alternatives to conventional NN algorithms, applicable across various machine learning tasks. Most importantly, the DMRG-like training algorithm provides direct access to the singular values along the virtual dimensions of the trainable MPOs of `TNLayers`, from which a measure of entanglement (correlation) between the features and model parameters can be computed.

Our TNN and DMRG-like algorithm suggest that tensor networks are closely related to neural networks. In fact, our approach opens the door for designing new numerical techniques for obtaining neural network representations of a quantum state and is a valuable tool to study the expressive power of quantum neural states. Furthermore, the ideas developed here can

further be extended to other deep learning architectures such as convolutional neural networks[57,58] and their training algorithm.

Last but not least, tensor network compression of trainable weights can have potential applications in providing compact representations of large models, such as large language models (LLMs). We have recently demonstrated how TN compression can be effectively used for efficient parameter reduction in LLMs, such as Llama-7B[59], resulting in a significantly more compact model while maintaining a high level of accuracy and efficiency.

Acknowledgements

S.S.J. acknowledges the support from Institute for Advanced Studies in Basic Sciences (IASBS), Donostia International Physics Center (DIPC) and Multiverse Computing.

Author contributions statement

All authors contributed equally to the manuscript.

Data Availability

The datasets used and/or analyzed during the current study available from the corresponding author on reasonable request.

References

1. Zhou, X., Gong, W., Fu, W. & Du, F. Application of deep learning in object detection. *Proc. - 16th IEEE/ACIS Int. Conf. on Comput. Inf. Sci. ICIS 2017* 631–634, DOI: 10.1109/ICIS.2017.7960069 (2017).
2. Sonoda, J. & Kimoto, T. Object identification form GPR images by deep learning. *Asia-Pacific Microw. Conf. Proceedings, APMC* **2018-November**, 1298–1300, DOI: 10.23919/APMC.2018.8617556 (2019).
3. Chang, Y. H., Chung, P. L. & Lin, H. W. Deep learning for object identification in ROS-based mobile robots. *Proc. 4th IEEE Int. Conf. on Appl. Syst. Innov. 2018, ICASI 2018* 66–69, DOI: 10.1109/ICASI.2018.8394348 (2018).
4. Li, S. *et al.* Deep learning for hyperspectral image classification: An overview. *IEEE Transactions on Geosci. Remote. Sens.* **57**, 6690–6709, DOI: 10.1109/TGRS.2019.2907932 (2019). 1910.12861.
5. Affonso, C., Rossi, A. L. D., Vieira, F. H. A. & de Carvalho, A. C. P. d. L. F. Deep learning for biological image classification. *Expert. Syst. with Appl.* **85**, 114–122, DOI: 10.1016/J.ESWA.2017.05.039 (2017).
6. Krizhevsky, A., Sutskever, I. & Hinton, G. E. ImageNet classification with deep convolutional neural networks. *Commun. ACM* **60**, 84–90, DOI: 10.1145/3065386 (2017).
7. Lathuiliere, S., Mesejo, P., Alameda-Pineda, X. & Horaud, R. A Comprehensive Analysis of Deep Regression. *IEEE Transactions on Pattern Analysis Mach. Intell.* **42**, 2065–2081, DOI: 10.1109/TPAMI.2019.2910523 (2020). 1803.08450.
8. Ramsundar, B. & Zadeh, R. B. *TensorFlow for deep learning : from linear regression to reinforcement learning* (O'Reilly Media, Inc., 2018).
9. Tian, K., Zhou, S. & Guan, J. DeepCluster: A General Clustering Framework Based on Deep Learning. *Lect. Notes Comput. Sci. (including subseries Lect. Notes Artif. Intell. Lect. Notes Bioinformatics)* **10535 LNAI**, 809–825, DOI: 10.1007/978-3-319-71246-8_49/COVER (2017).
10. Min, E. *et al.* A Survey of Clustering with Deep Learning: From the Perspective of Network Architecture. *IEEE Access* **6**, 39501–39514, DOI: 10.1109/ACCESS.2018.2855437 (2018).
11. Aljalbout, E., Golkov, V., Siddiqui, Y. & Cremers, D. Clustering with deep learning: Taxonomy and new methods. *ArXiv* **abs/1801.07648** (2018).
12. Yu, D., Seltzer, M. L., Li, J., Huang, J. T. & Seide, F. Feature Learning in Deep Neural Networks - Studies on Speech Recognition Tasks. *1st Int. Conf. on Learn. Represent. ICLR 2013 - Conf. Track Proc.* DOI: 10.48550/arxiv.1301.3605 (2013). 1301.3605.
13. Jing, L. & Tian, Y. Self-Supervised Visual Feature Learning with Deep Neural Networks: A Survey. *IEEE Transactions on Pattern Analysis Mach. Intell.* **43**, 4037–4058, DOI: 10.1109/TPAMI.2020.2992393 (2021). 1902.06162.
14. Dehaene, S. & Changeux, J. P. Reward-dependent learning in neuronal networks for planning and decision making. *Prog. Brain Res.* **126**, 217–229, DOI: 10.1016/S0079-6123(00)26016-0 (2000).
15. Hill, T., Marquez, L., O'Connor, M. & Remus, W. Artificial neural network models for forecasting and decision making. *Int. J. Forecast.* **10**, 5–15, DOI: 10.1016/0169-2070(94)90045-0 (1994).

16. Bedolla, E., Padierna, L. C. & Castañeda-Priego, R. Machine learning for condensed matter physics. *J. Physics: Condens. Matter* **33**, 053001, DOI: 10.1088/1361-648X/ABB895 (2020). 2005.14228.

17. Karniadakis, G. E. *et al.* Physics-informed machine learning. *Nat. Rev. Phys. 2021 3:6* **3**, 422–440, DOI: 10.1038/s42254-021-00314-5 (2021).

18. Mehta, P. *et al.* A high-bias, low-variance introduction to Machine Learning for physicists. *Phys. Reports* **810**, 1–124, DOI: 10.1016/j.physrep.2019.03.001 (2019). 1803.08823.

19. Sharir, O., Shashua, A. & Carleo, G. Neural tensor contractions and the expressive power of deep neural quantum states. *Phys. Rev. B* **106**, 205136, DOI: 10.1103/PhysRevB.106.205136 (2022).

20. Pescia, G., Han, J., Lovato, A., Lu, J. & Carleo, G. Neural-network quantum states for periodic systems in continuous space. *Phys. Rev. Res.* **4**, 023138, DOI: 10.1103/PHYSREVRESEARCH.4.023138/FIGURES/5/MEDIUM (2022). 2112.11957.

21. Gutiérrez, I. L. & Mendl, C. B. Real time evolution with neural-network quantum states. *Quantum* **6**, 627, DOI: 10.22331/q-2022-01-20-627 (2022). 1912.08831v4.

22. Pescia, G., Han, J., Lovato, A., Lu, J. & Carleo, G. Neural-network quantum states for periodic systems in continuous space. *Phys. Rev. Res.* **4**, 023138, DOI: 10.1103/PHYSREVRESEARCH.4.023138/FIGURES/5/MEDIUM (2022). 2112.11957.

23. Wu, Y., Yao, J., Zhang, P. & Zhai, H. Expressivity of quantum neural networks. *Phys. Rev. Res.* **3**, L032049, DOI: 10.1103/PHYSREVRESEARCH.3.L032049/FIGURES/2/MEDIUM (2021). 2101.04273.

24. Carrasquilla, J. & Melko, R. G. Machine learning phases of matter. *Nat. Phys. 2017 13:5* **13**, 431–434, DOI: 10.1038/nphys4035 (2017). 1605.01735.

25. Greplova, E. *et al.* Unsupervised identification of topological phase transitions using predictive models. *New J. Phys.* **22**, 045003, DOI: 10.1088/1367-2630/AB7771 (2020).

26. Tanaka, A. & Tomiya, A. Detection of Phase Transition via Convolutional Neural Networks. *http://dx.doi.org/10.7566/JPSJ.86.063001* **86**, DOI: 10.7566/JPSJ.86.063001 (2017). 1609.09087.

27. Zhang, Y., Melko, R. G. & Kim, E. A. Machine learning Z2 quantum spin liquids with quasiparticle statistics. *Phys. Rev. B* **96**, 245119, DOI: 10.1103/PHYSREVB.96.245119/FIGURES/9/MEDIUM (2017).

28. Orus, R. R. A practical introduction to tensor networks: Matrix product states and projected entangled pair states. *Annals Phys.* **349**, 117–158, DOI: 10.1016/J.AOP.2014.06.013 (2014). 1306.2164.

29. Verstraete, F., Murg, V. & Cirac, J. Matrix product states, projected entangled pair states, and variational renormalization group methods for quantum spin systems. *Adv. Phys.* **57**, 143–224, DOI: 10.1080/14789940801912366 (2008).

30. Stoudenmire, E. & Schwab, D. J. Supervised learning with tensor networks. In Lee, D., Sugiyama, M., Luxburg, U., Guyon, I. & Garnett, R. (eds.) *Advances in Neural Information Processing Systems*, vol. 29 (Curran Associates, Inc., 2016).

31. Patel, R. G. *et al.* Quantum-inspired tensor neural networks for partial differential equations. *ArXiv* **abs/2208.02235** (2022).

32. Strashko, A. & Stoudenmire, E. M. Generalization and Overfitting in Matrix Product State Machine Learning Architectures, DOI: 10.48550/arXiv.2208.04372 (2022). ArXiv:2208.04372 [quant-ph].

33. Barratt, F., Dborin, J. & Wright, L. Improvements to Gradient Descent Methods for Quantum Tensor Network Machine Learning, DOI: 10.48550/arXiv.2203.03366 (2022). ArXiv:2203.03366 [cond-mat, physics:quant-ph].

34. Wang, J., Roberts, C., Vidal, G. & Leichenauer, S. Anomaly detection with tensor networks. *ArXiv* **abs/2006.02516** (2020).

35. Stoudenmire, E. M. Learning relevant features of data with multi-scale tensor networks. *Quantum Sci. Technol.* **3**, 034003, DOI: 10.1088/2058-9565/aaba1a (2018).

36. Panagakis, Y. *et al.* Tensor Methods in Computer Vision and Deep Learning. *Proc. IEEE* **109**, 863–890, DOI: 10.1109/jproc.2021.3074329 (2021). 2107.03436.

37. Bahri, M., Panagakis, Y. & Zafeiriou, S. Robust Kronecker Component Analysis. *IEEE Transactions on Pattern Analysis Mach. Intell.* **41**, 2365–2379, DOI: 10.1109/TPAMI.2018.2881476 (2019). 1801.06432.

38. Kossaifi, J., Khanna, A., Lipton, Z., Furlanello, T. & Anandkumar, A. Tensor Contraction Layers for Parsimonious Deep Nets. *IEEE Comput. Soc. Conf. on Comput. Vis. Pattern Recognit. Work.* **2017-July**, 1940–1946, DOI: 10.1109/CVPRW.2017.243 (2017). 1706.00439.

39. Kossaifi, J. *et al.* Tensor Regression Networks. *J. Mach. Learn. Res.* **21**, 1–21 (2020).

40. Martin-Ramiro, P. *et al.* Boosting defect detection in manufacturing using tensor convolutional neural networks. *In prepration* (2022).

41. White, S. R. Density matrix formulation for quantum renormalization groups. *Phys. Rev. Lett.* **69**, 2863–2866, DOI: 10.1103/PhysRevLett.69.2863 (1992).

42. White, S. R. Density-matrix algorithms for quantum renormalization groups. *Phys. Rev. B* **48**, 10345–10356, DOI: 10.1103/PhysRevB.48.10345 (1993).

43. Orus, R. & Vidal, G. Infinite time-evolving block decimation algorithm beyond unitary evolution. *Phys. Rev. B - Condens. Matter Mater. Phys.* **78**, 155117, DOI: 10.1103/PhysRevB.78.155117 (2008). 0711.3960.

44. Vidal, G. Efficient classical simulation of slightly entangled quantum computations. *Phys. Rev. Lett.* **91**, 147902, DOI: 10.1103/PhysRevLett.91.147902 (2003). 0301063.

45. Vidal, G. Efficient simulation of one-dimensional quantum many-body systems. *Phys. Rev. Lett.* **93**, 040502–1, DOI: 10.1103/PhysRevLett.93.040502 (2004). 0310089.

46. Novikov, A., Podoprikhin, D., Osokin, A. & Vetrov, D. P. Tensorizing neural networks. In Cortes, C., Lawrence, N., Lee, D., Sugiyama, M. & Garnett, R. (eds.) *Advances in Neural Information Processing Systems*, vol. 28 (Curran Associates, Inc., 2015).

47. Gao, Z. F. *et al.* Compressing deep neural networks by matrix product operators. *Phys. Rev. Res.* **2**, 023300, DOI: 10.1103/PHYSREVRESEARCH.2.023300/FIGURES/3/MEDIUM (2020). 1904.06194.

48. Liu, P. *et al.* Enabling Lightweight Fine-tuning for Pre-trained Language Model Compression based on Matrix Product Operators. *ACL-IJCNLP 2021 - 59th Annu. Meet. Assoc. for Comput. Linguist. 11th Int. Jt. Conf. on Nat. Lang. Process. Proc. Conf.* 5388–5398, DOI: 10.18653/V1/2021.ACL-LONG.418 (2021). 2106.02205.

49. Sun, X., Gao, Z. F., Lu, Z. Y., Li, J. & Yan, Y. A Model Compression Method with Matrix Product Operators for Speech Enhancement. *IEEE/ACM Transactions on Audio Speech Lang. Process.* **28**, 2837–2847, DOI: 10.1109/TASLP.2020.3030495 (2020). 2010.04950.

50. Qi, J., Yang, C.-H. H., Chen, P.-Y. & Tejedor, J. Exploiting Low-Rank Tensor-Train Deep Neural Networks Based on Riemannian Gradient Descent With Illustrations of Speech Processing. *IEEE/ACM Transactions on Audio, Speech, Lang. Process.* **31**, 633–642, DOI: 10.1109/TASLP.2022.3231714 (2023).

51. Qi, J., Yang, C.-H. H., Chen, P.-Y. & Hsieh, M.-H. Theoretical error performance analysis for variational quantum circuit based functional regression. *npj Quantum Inf.* **9**, 4, DOI: 10.1038/s41534-022-00672-7 (2023).

52. Chen, S. Y.-C., Huang, C.-M., Hsing, C.-W. & Kao, Y.-J. An end-to-end trainable hybrid classical-quantum classifier. *Mach. Learn. Sci. Technol.* **2**, 045021, DOI: 10.1088/2632-2153/ac104d (2021).

53. Abadi, M. *et al.* TensorFlow: Large-scale machine learning on heterogeneous systems (2015). Software available from tensorflow.org.

54. Paszke, A. *et al.* Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*, 8024–8035 (Curran Associates, Inc., 2019).

55. Margossian, C. C. A review of automatic differentiation and its efficient implementation. *Wiley Interdiscip. Rev. Data Min. Knowl. Discov.* **9**, e1305, DOI: 10.1002/WIDM.1305 (2019). 1811.05031.

56. Baydin, A. G., Pearlmutter, B. A., Radul, A. A. & Siskind, J. M. Automatic Differentiation in Machine Learning: a Survey. *J. Mach. Learn. Res.* **18**, 1–43 (2018).

57. Martin-Ramiro, P., de la Maza, U. S., Singh, S., Orus, R. & Mugel, S. Boosting Defect Detection in Manufacturing using Tensor Convolutional Neural Networks, DOI: 10.48550/arXiv.2401.01373 (2024). ArXiv:2401.01373 [quant-ph].

58. Singh, S., Jahromi, S. S. & Orus, R. Tensor network compressibility of convolutional models, DOI: 10.48550/arXiv.2403.14379 (2024). ArXiv:2403.14379 [quant-ph].

59. Tomut, A. *et al.* CompactifAI: Extreme Compression of Large Language Models using Quantum-Inspired Tensor Networks, DOI: 10.48550/arXiv.2401.14109 (2024). ArXiv:2401.14109 [quant-ph].